ECE 350: Real-Time Operating Systems Winter 2027

Lecture 31 — Live Kernel Patching

Jeff Zarnett *2026-07-10*

Can't Stop, Won't Stop

When people are counting on a system – particularly life and safety-critical ones – it might be difficult to have a maintenance window for it. Let's say that we cannot, or at least do not want to, have a system restart for the purposes of updating the kernel. It is perhaps not the right choice to disrupt a long-running task, but also not great to leave security vulnerabilities unaddressed. We need a third option: change the kernel without a reboot.

Many modern software projects allow for some sort of staged or transitional rollout where you start sending new requests to the new system and let the old system finish out old requests before you shut it down. But the kernel of the operating system doesn't permit this sort of thing,

Kernels do have the ability to load new modules; this is what we do with device drivers on a regular basis. So if what we need is to unload and reload a kernel module, that's not difficult. What we're talking about here, though, is the idea of changing the kernel's main functionality. And doing it live. Can we load some code into the kernel and have it replace existing things?

In practice, this is a little bit like the kernel doing a surgery on itself. There is a precedent of sorts in the real world: a Soviet doctor who was stuck in Novolazarevskaya Station, Antarctica needed to do a surgery to remove his own appendix[1]. That was perhaps a little risky, but what choice did he have? It was 1961 in Antarctica. Still, it's doable if we adhere to some restrictions and do it well.

The basic plan has a few steps. Step one is validating that we can even make this change at all; changes to functions is fine, but data structures cannot be changed on the fly. Then, prepare the patch; making the compiled binary code that we intend to replace some part of the existing kernel. Then we need to make sure that it's safe to make the swap (details below in the implementation description) and execute the replacement. After that, future system calls then run the new code. This high-level view makes it seem easy, but the challenge is in the details, particularly the part about making sure it's safe to make the swap.

The good news is that many patches are relatively straightforward in terms of the size of the change. In fact, the Kernel docs around this say: "Many fixes do not change the semantic of the modified functions. For example, they add a `NULL` pointer or a boundary check, fix a race by adding a missing memory barrier, or add some locking around a critical section. Most of these changes are self contained and the function presents itself the same way to the rest of the system."[2] In other words, a we're changing a few lines within the function, but the function's main logic, purpose, signature, return type, etc. do not change. This makes the size of the problem a lot more manageable than wholesale replacement. Though, in theory, the patching mechanism can be used to do a larger change than just one of these "typical" cases. (Ask me some time about using the Ruby monkey-patching mechanism on Rails `ActiveRecord` to parameter-bind large arrays of IDs in SQL `IN(...)` queries.)

As you may imagine, the consequences of getting this wrong are potentially severe. It could lead to a crash (kernel panic), but it could also lead to silent corruption of the system and its data. Or we could introduce some security vulnerabilities or data leakage, which is rather the opposite of the purpose.

[1] `https://en.wikipedia.org/wiki/Leonid_Rogozov`
[2] `https://docs.kernel.org/livepatch/livepatch.html`

Patching with `kpatch` and `kGraft`

If you're familiar with Linux and open source software in general, it won't surprise you to know that there were two competing approaches to this that came out around the same time in 2014: `kpatch` and `kGraft`. The final version of live patching in the kernel took some parts of each of these, but we can examine each of them individually and then see how they came together in the end.

The `kpatch` [Poi14] approach...

The `kGraft` [Wal14] approach...

References

[Poi14] Josh Poimboeuf. Introducing kpatch: Dynamic kernel patching, 2014. Online; accessed 10-July-2026. URL: `https://www.redhat.com/en/blog/introducing-kpatch-dynamic-kernel-patching`.

[Wal14] Jack Wallen. SUSE Enterprise Linux Live Kernel Patching with Open Source kGraft, 2014. Online; accessed 10-July-2026. URL: `https://www.linux.com/news/suse-enterprise-linux-live-kernel-patching-open-source-kgraft/`.